



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第7章 动态规划3—算法案例

梁宏涛

lht@qust.edu.cn

周二&周四上午

1/71

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

2/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

3/182

7.7 0/1背包问题

 **问题描述：**有 n 个物品，物品编号为 $0 \sim n-1$ ，重量为 $\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。现在从这些物品中选取若干物品装入该背包的方案，每个物品要么选中要么不选中，要求选中的物品总重量不超过背包容量并且具有最大的价值。

解

- 设置二维动态规划数组 **dp**。
- **dp[i][r]** 表示在物品 $0 \sim i-1$ （共 i 个物品）中选择物品并且背包容量为 r （ $0 \leq r \leq W$ ）时最大价值，或者说只考虑前 i 个物品并且背包容量为 r 时最大价值。

当考虑物品 $0 \sim i-2$ 之后，已经求出相应的 dp 元素，现在考虑物品 $i-1$ ，分为两种情况：

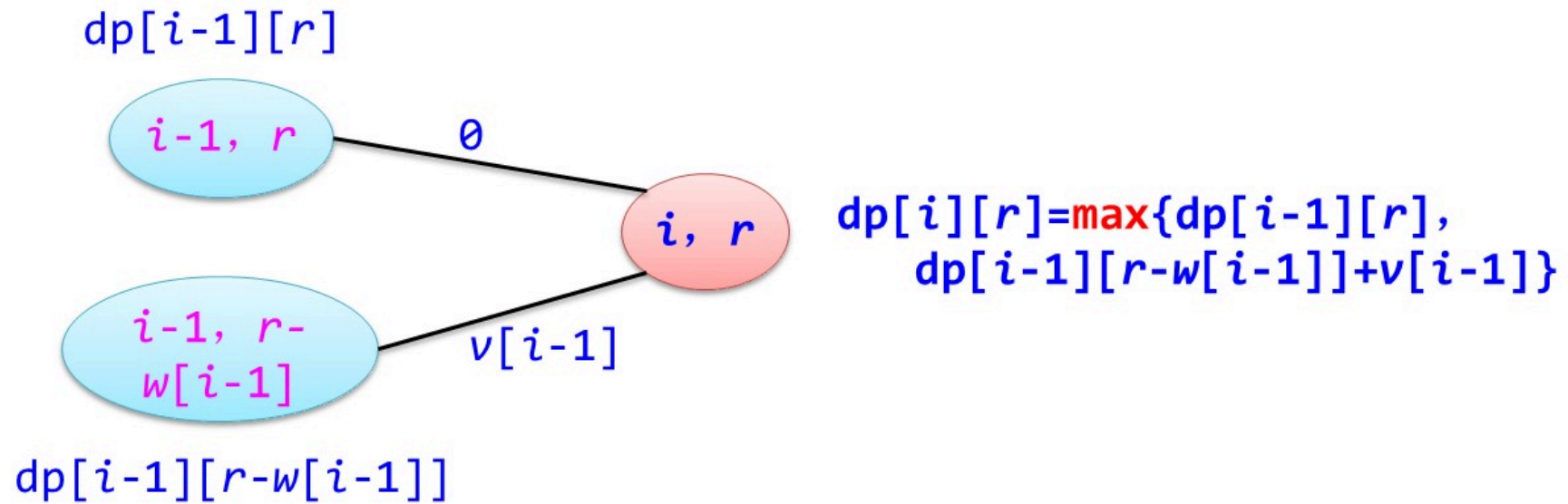
1. 若 $r < w[i-1]$ ，说明物品 $i-1$ 放不下，此时等效于只考虑前 $i-1$ 个物品并且背包容量为 r 时的最大价值，所以有 $dp[i][r] = dp[i-1][r]$ 。

2. 若 $r \geq w[i-1]$, 说明物品 $i-1$ 能够放入背包, 又有两种子情况:

- 不选择物品 $i-1$ 即不将物品 $i-1$ 放入背包, 等同于情况1。
- 选择物品 $i-1$ 即将物品 $i-1$ 放入背包, 这样消耗了 $w[i-1]$ 的背包容量, 获取了 $v[i-1]$ 的价值, 那么留给前 $i-1$ 个物品的背包容量就只有 $r-w[i-1]$ 了, 此时的最大价值为

$$dp[i-1][r-w[i-1]]+v[i-1]。$$

状态转移图



状态转移方程

$dp[i][0]=0$ (没有装入任何物品, 总价值为0)

边界情况

$dp[0][r]=0$ (没有考虑任何物品, 总价值为0)

边界情况

$dp[i][r]=dp[i-1][r]$

当 $r < w[i-1]$ 时, 物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]]+v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

在求出 dp 数组后 $dp[n][W]$ 元素就是答案。

主观题 10分

0/1背包问题:

$W_{\max}=5$; 重量为 $\{1, 3, 5, 1, 2\}$, 价值为 $\{3, 2, 7, 2, 9\}$, 求解DP, 拍照上传。

$dp[i][0]=0$ (没有装入任何物品, 总价值为0)

边界情况

$dp[0][r]=0$ (没有考虑任何物品, 总价值为0)

边界情况

$dp[i][r]=dp[i-1][r]$

当 $r < w[i-1]$ 时, 物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]]+v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

在求出dp数组后 $dp[n][W]$ 元素就是答案。

```

vector<vector<int>> dp;                                //二维动态规划数组
int knap(int w[],int v[],int n,int W) {
    dp=vector<vector<int>>(n+1,vector<int>(W+1));
    for (int i=0;i<=n;i++) dp[i][0]=0;    //置边界条件
    for (int r=0;r<=W;r++) dp[0][r]=0;    //置边界条件
    for (int i=1;i<=n;i++) {
        for (int r=0;r<=W;r++) {
            if (r<w[i-1])
                dp[i][r]=dp[i-1][r];
            else
                dp[i][r]=max(dp[i-1][r],dp[i-1][r-w[i-1]]
                    +v[i-1]);
        }
    }
    return dp[n][W];
}

```

10/182

 **knap算法分析**：该算法中包含两重**for**循环，所以时间复杂度为 $O(nW)$ ，空间复杂度为 $O(nW)$ 。

★表面上看其时间函数是 n 的多项式，但实际上并不是真正多项式级的算法。如果 W 不是太大，则 nW 要远好于一个 n 的指数级。如果 W 非常大，或者为很大的实数，则该动态规划方法不可行，所以说这样的算法是伪多项式时间的算法。

当求出 dp 数组后，如何推导出一个解向量 $x = (x_0, x_1, \dots, x_{n-1})$ 呢？从前面状态转移方程的后两行看出：

- 若 $dp[i][r] = dp[i-1][r]$ ，表示物品 $i-1$ 放不下或者不放入物品 $i-1$ 的情况，总之不选择物品 $i-1$ 。也就是说当前 dp 元素等于上方元素，不选择对应的物品（物品 $i-1$ ），并跳到上方位置。
- 否则一定有 $dp[i][r] \neq dp[i-1][r]$ 成立，表示选择物品 $i-1$ 。也就是说当前 dp 元素不等于上方元素，选择对应的物品，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置。

$dp[i][r] = dp[i-1][r]$

当 $r < w[i-1]$ 时，物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

程序
验证

$n=5$, $w=\{2, 2, 6, 5, 4\}$, $v=\{6, 3, 5, 4, 6\}$, $W=10$ 。



dp:

0	0	0	0	0	0	0	0	0	0	0
0	0	6	6	6	6	6	6	6	6	6
0	0	6	6	9	9	9	9	9	9	9
0	0	6	6	9	9	9	9	11	11	14
0	0	6	6	9	9	9	10	11	13	14
0	0	6	6	9	9	12	12	15	15	15

求解结果<最优方案>
选取的物品: 0 1 4
总价值=15

$i \backslash r$	0	1	2	3	4	5	6	7	8	9	10
$w_0=2$	0	0	0	0	0	0	0	0	0	0	0
$w_1=2$	1	0	0	6	6	6	6	6	6	6	6
$w_2=6$	2	0	0	6	6	9	9	9	9	9	9
$w_3=5$	3	0	0	6	6	9	9	9	11	11	14
$w_4=4$	4	0	0	6	6	9	9	9	10	11	13
5	0	0	6	6	9	9	12	12	15	15	15

选择物品4
选择物品1
选择物品0

- 若 $dp[i][r] = dp[i-1][r]$ ，当前 dp 元素等于上方元素，不选择物品 $i-1$ ，并跳到上方位置，即 $dp[i][r] \Rightarrow dp[i-1][r]$ 。
- 否则当前 dp 元素不等于上方元素，选择对应的物品 $i-1$ ，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置，即 $dp[i][r] \Rightarrow dp[i-1][r-w[i-1]]$ 。

14/182

```

void getx(int w[],int n,int W) { //回推求一个最优方案
    vector<int> x(n,0);          //解向量
    int i=n,r=W;
    while (i>=1) {
        if (dp[i][r]!=dp[i-1][r]) {
            x[i-1]=1;           //选取物品i-1
            r=r-w[i-1];
        }
        else x[i-1]=0;          //不选取物品i-1
        i--;
    }
    printf("    选择的物品:");
    for(int i=0;i<n;i++) {
        if(x[i]==1) printf(" 物品%d",i);
    }
    printf("\n");
}

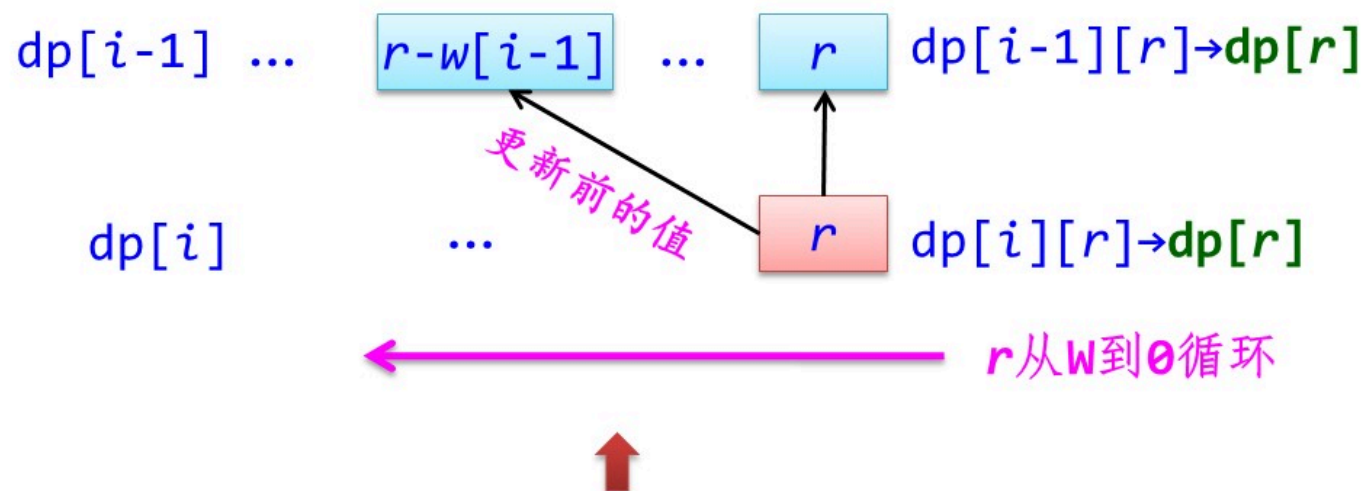
```

15/182

算法空间优化*

- $dp[MAXN][MAXW]$ 改为一维数组 $dp[MAXW]$ 。

$dp[i-1][r-w[i-1]] \rightarrow dp[r-w[i-1]]$



$dp[i][r] = dp[i-1][r]$
 $dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$

16/182

```

int knap1(int w[],int v[],int n,int W) {    //改进算法
    vector<int> dp(W+1,0);                //一维动态规划数组
    for (int i=1;i<=n;i++) {
        for (int r=W;r>=0;r--) { ?        //r按0到W的逆序（重点）
            if (r<w[i-1])
                dp[r]=dp[r];
            else
                dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
        }
    }
    return dp[W];
}

```

17/182

可以等价地改为如下

```
int knap2(int w[],int v[],int n,int W) {    //改进算法
    vector<int> dp(W+1,0);                //一维动态规划数组
    for (int i=1;i<=n;i++) {
        for (int r=W;r>=w[i-1];r--) //r按w[i-1]到W的逆序（重点）
            dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
    }
    return dp[W];
}
```

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

19/182

7.8*完全背包和多重背包问题

7.8.1 完全背包问题

- 有 n 种重量和价值分别为 w_i 、 v_i ($0 \leq i < n$) 的物品，从这些物品中挑选总重量不超过 W 的物品，每种物品可以挑选任意多件。
- 求挑选物品的最大价值。

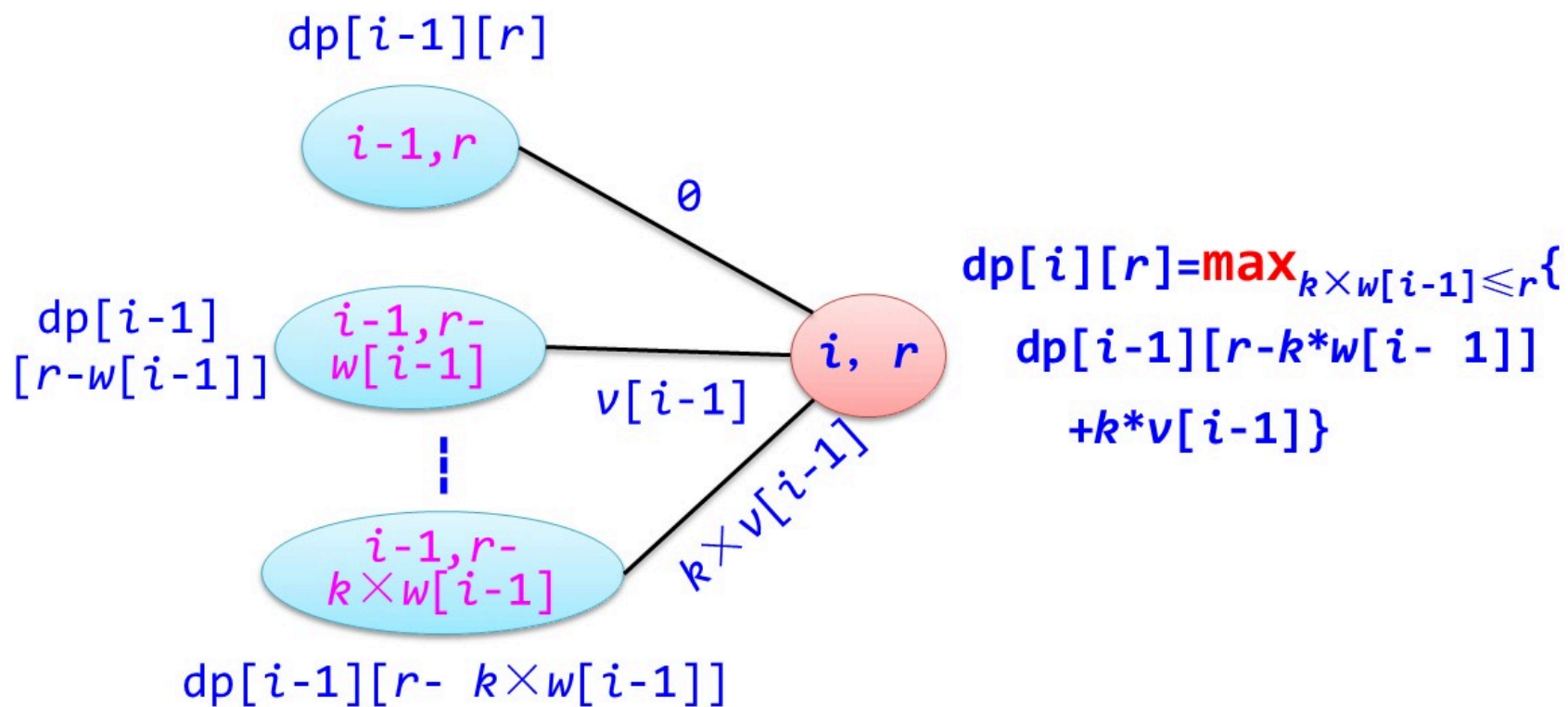
20/182

解

- 设置动态规划二维数组 dp ， $dp[i][r]$ 表示从物品 $0 \sim i-1$ （共 i 种物品）中选出重量不超过 r 的物品的最大总价值。
- 显然有 $dp[i][0]=0$ （背包不能装入任何物品时，总价值为 0 ）， $dp[0][j]=0$ （没有任何物品可装入时，总价值为 0 ），将它们作为边界情况，为此一次性将 dp 数组初始化为 0 。

21/182

一般情况:



22/182

- 另外设置二维数组 **fk**，其中 **fk[i][r]** 存放 **dp[i][r]** 得到最大值时物品 **i-1** 挑选的件数。考虑物品 **i-1**，可以选择 **0** 到 **k** ($k \times w[i-1] \leq r$) 次。



状态转移方程

$$\begin{aligned} dp[i][r] &= \max_{k \times w[i-1] \leq r} \{ dp[i-1][r - k \times w[i-1]] + k \times v[i-1] \} \\ fk[i][r] &= k; \end{aligned}$$

物品 **i-1** 取 **k** 件

$n=3, W=7, w=(3, 4, 2), v=(4, 5, 3)$

$i \backslash j$	0	1	2	3	4	5	6	7
0	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]
1	0[0]	0[0]	0[0]	4[1]	4[1]	4[1]	8[2]	8[2]
2	0[0]	0[0]	0[0]	4[0]	5[1]	5[1]	8[0]	9[1]
3	0[0]	0[0]	3[1]	4[0]	6[2]	7[1]	9[3]	10[2]

最优方案：物品2挑选2件，物品1挑选0件，物品0挑选1件。

```

vector<vector<int>> dp;           //二维动态规划数组
vector<vector<int>> fk;         //物品件数

int completeknap(int w[],int v[],int n,int W) {
    dp=vector<vector<int>>(n+1,vector<int>(W+1,0));
    fk=vector<vector<int>>(n+1,vector<int>(W+1,0));
    for (int i=1;i<=n;i++) {
        for (int r=0;r<=W;r++) {
            for (int k=0;k*w[i-1]<=r;k++) {
                if (dp[i][r]<dp[i-1][r-k*w[i-1]]+k*v[i-1]) {
                    dp[i][r]=dp[i-1][r-k*w[i-1]]+k*v[i-1];
                    fk[i][r]=k;           //物品i-1取k件
                }
            }
        }
    }
    return dp[n][W];
}

```

25/182

```

void getx(int w[],int n,int W) {    //回推求一个最优方案
    int i=n,r=W;
    while (i>=1) {
        printf("    选择物品%d共%d件\n",i-1,fk[i][r]);
        r-=fk[i][r]*w[i-1];        //剩余重量
        i--;
    }
    printf("\n");
}

```

【算法分析】 **completeKnap**算法中包含三重循环，**k**的循环最坏可能从**0**到**W**，所以算法的时间复杂度为 **$O(nW^2)$** 。

算法时间优化

- 将完全背包问题转换为这样的0/1背包问题，物品 i 出现 $\lfloor W/w[i] \rfloor$ 次。
- 例如对于完全背包问题 $n=3$, $W=7$, $w=(3, 4, 2)$, $v=(4, 5, 3)$, 物品0最多取 $W/3=2$ 次, 物品1最多取 $W/4=1$ 次, 物品2最多取 $W/2=3$ 次, 对应的0/1背包问题是 $W=7$, $n=6$, $w=(3, 3, 4, 2, 2, 2)$, $v=(4, 4, 5, 3, 3, 3)$ 。
- 后者的最大价值与前者是相同的。

27/182

```

int completeknapsack1(int w[],int v[],int n,int W) {
//时间改进算法
    dp=vector<vector<int>>(n+1,vector<int>(W+1,0));
    for (int i=1;i<=n;i++) {
        for (int r=0;r<=W;r++) {
            if (r<w[i-1])           //物品i-1放不下
                dp[i][r]=dp[i-1][r];
            else                     //在不选择和选择物品i-1（多次）中求最大值
                dp[i][r]=max(dp[i-1][r],dp[i][r-w[i-1]]+v[i-1]);
        }
    }
    return dp[n][W];               //返回总价值
}

```

【算法分析】 上述算法中包含两重循环，所以算法的时间复杂度为 $O(nW)$ 。

28/182

算法空间优化

```
int completeknap2(int w[],int v[],int n,int W) {  
    vector<int> dp(W+1,0);           //一维动态规划数组  
    for (int i=1;i<=n;i++) {  
        for (int r=w[i-1];r<=W;r++) //r从w[i-1]到W遍历  
            dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);  
    }  
    return dp[W];  
}
```

```
int knap2(int w[],int v[],int n,int W) {    //0/1背包  
    vector<int> dp(W+1,0);           //一维动态规划数组  
    for (int i=1;i<=n;i++) {  
        for (int r=W;r>=w[i-1];r--) //r按w[i-1]到W的逆序  
            dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);  
    }  
    return dp[W];  
}
```

} 差别

29/182



【例7.3】零钱兑换（LeetCode322★★）。给定一个含 n （ $1 \leq n \leq 12$ ）个整数的数组`coins`，表示不同面额的硬币（ $1 \leq \text{coins}[i] \leq 2^{31}-1$ ），以及一个表示总金额的整数`amount`（ $0 \leq \text{amount} \leq 10^4$ ）。设计一个算法求可以凑成总金额所需的最少的硬币个数，如果没有任何一种硬币组合能组成总金额则返回-1。可以认为每种硬币的数量是无限的。

例如，`coins={1, 2, 5}`，`amount=11`，对应的硬币组合是1, 5, 5，答案为3。

要求设计如下成员函数：

```
int coinChange(vector<int>& coins, int amount) {}
```

30/182

解

- 由于每种硬币的数量是无限的，该问题转换为完全背包问题，只是这里求最少的硬币个数，相当于每个硬币的价值为**1**，并且将**max**改为**min**。
- 采用改进的完全背包动态规划算法，设置一维动态规划数组**dp**，**dp[r]**表示总金额为**r**的最少的硬币个数。
- 另外考虑特殊情况，将**dp**所有元素初始化为 ∞ ，当最后出现**dp[amount]**为 ∞ ，说明没有任何一种硬币组合能组成**amount**金额，返回**-1**。

31/182

```

class Solution {
    const int INF=0x3f3f3f3f;           //表示∞
public:
    int coinChange(vector<int>& coins, int amount) {
        int n=coins.size();
        if (amount==0) return 0;
        int dp[amount+1];               //一维动态规划数组
        memset(dp,0x3f,sizeof(dp));     //置边界情况
        dp[0]=0;
        for (int i=1;i<=n;i++) {
            for (int r=1;r<=amount;r++) {
                if (r<=coins[i-1])
                    dp[r]=min(dp[r],dp[r-coins[i-1]]+1);
            }
        }
        return dp[amount]==INF?-1:dp[amount];
    }
};

```

32/182



执行用时为**56ms**，内存消耗为**9.7MB**。

33/182

7.8.2 多重背包问题

- 多重背包问题是在完全背包问题基础上增加了每种物品的数量限制，有 n 种重量和价值分别为 w_i 、 v_i ($0 \leq i < n$) 的物品，每种物品的数目为 s_i ，从这些物品中挑选总重量不超过 W 的物品，求挑选物品的最大价值。同样可以将多重背包问题转换为0/1背包问题。
- 例如对于多重背包问题 $n=3$, $W=7$, $w=(3, 4, 2)$, $v=(4, 5, 3)$, $s=(2, 2, 1)$ ，对应的0/1背包问题是 $W=7$, $n=5$, $w=(3, 3, 4, 4, 2)$, $v=(4, 4, 5, 5, 3)$ ，后者的最大价值与前者是相同的。

【例7.4】 为了庆祝某班在学校运动会上取得第一名成绩，班主任决定开一场庆功会，为此拨款购买奖品，期望拨款金额能够购买最大价值的奖品。给定希望购买奖品的种数 n 和拨款金额 W ，每种奖品的价格数组 w 、价值数组 v 和能够购买的最大数目数组 s （注意这里价格和价值是不同的概念）。设计一个算法求购买奖品的最大价值。

例如， $n=3$ ， $W=7$ ， $w=\{3, 4, 2\}$ ， $v=\{4, 5, 3\}$ ， $s=\{2, 2, 1\}$ ，答案是9，购买物品0和物品1各1件。

解

- 设置动态规划二维数组 dp ，其中 $dp[i][r]$ 表示从物品 $0 \sim i-1$ （共 i 个物品）中选出价格不超过 r 并且满足物品数量限制的最大总价值。
- 将 dp 数组所有元素初始化为 0 。对应的状态转移方程如下：

$$dp[i][r] = \max_{0 \leq k \leq s[i-1] \ \&\& \ k \times w[i-1] \leq r} \{dp[i-1][r - k \times w[i-1]] + k \times v[i-1]\}$$

- 求出 dp 数组后， $dp[n][W]$ 便是最大价值。

36/182

```

int solve(int n,int W,vector<int>&w,
          vector<int>&v,vector<int>&s) {
    vector<vector<int>> dp(n+1,vector<int>(W+1,0));
    //二维动态规划数组
    for (int i=1;i<=n;i++) {
        for (int r=0;r<=W;r++) {
            for (int k=0;k<=s[i-1];k++) {
                if(k*w[i-1]<=r)
                    dp[i][r]=max(dp[i][r],
                                   dp[i-1][r-k*w[i-1]]+k*v[i-1]);
            }
        }
    }
    return dp[n][W];
}

```

37/182

采用滚动数组方式，将`dp[i][r]`改为`dp[r]`:

```
int solve1(int n,int W,vector<int>&w,
           vector<int>&v,vector<int>&s) {
    vector<int> dp(W+1,0);
    for(int i=1;i<=n;i++) {
        for(int r=W;r>=0;r--) {           //r从W到0循环
            for(int k=0;k<=s[i-1] && k*w[i-1]<=r;k++) {
                dp[r]=max(dp[r],dp[r-k*w[i-1]]+k*v[i-1]);
            }
        }
    }
    return dp[W];
}
```

38/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

39/182

7.9 扔鸡蛋问题



 **问题描述：**扔鸡蛋问题（LeetCode1884★★）。

给定两枚相同的鸡蛋和一栋 n （ $1 \leq n \leq 1000$ ）层楼的建筑（楼层编号为 $1 \sim n$ ）。已知存在楼层 f （ $0 \leq f \leq n$ ），任何从高于 f 的楼层落下的鸡蛋都会碎，从 f 楼层或比它低的楼层落下的鸡蛋都不会碎。每次操作是取一枚没有碎的鸡蛋并把它从任一楼层 x 扔下（满足 $1 \leq x \leq n$ ），如果鸡蛋碎了就不能再次使用它，如果某枚鸡蛋扔下后没有摔碎则可以在之后的操作中重复使用这枚鸡蛋。求要确定 f 确切的值的最小操作次数是多少？

40/182

例如, $n=10$, 一种最优的策略是将第一枚鸡蛋从4楼扔下, 如果碎了, 那么 f 在0和4之间, 将第二枚从1楼扔下, 然后每扔一次上一层楼, 在3次内找到 f , 总操作次数 $=1+3=4$ 。如果第一枚鸡蛋没有碎, 相当于在楼层5~10共6层的楼中用两枚鸡蛋找到 f , 将第一枚鸡蛋从7楼扔下, 如果碎了, 那么 f 在5和7之间, 将第二枚从5楼扔下, 然后每扔一次上一层楼, 在2次内找到 f , 总操作次数 $=1+1+2=4$ 。如果从7楼扔下第一枚鸡蛋没有碎, 相当于在楼层8~10共3层的楼中用两枚鸡蛋找到 f , 其结果为2次, 总操作次数 $=1$ (4层扔一次) $+1$ (7层扔一次) $+2=4$, 所以答案为4。

要求设计如下成员函数:

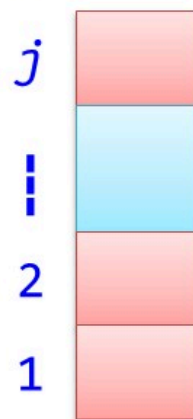
```
int twoEggDrop(int n) { }
```

解

设计二维动态规划数组 dp ，其中 $dp[i][j]$ 表示有 $i+1$ 枚鸡蛋时，验证高度为 j 层楼需要的最少操作次数，分为 $i=0$ 和 $i=1$ 的情况：

(1) 当 $i=0$ 时表示只剩一枚鸡蛋，此时需要从楼层1开始逐层验证才能确保获取确切的 f 值（ f 为扔鸡蛋的最优楼层），因此对于任意的 j （ $1 \leq j \leq n$ ）都有 $dp[0][j]=j$ ，如图(a)所示。

$dp[0][j]$

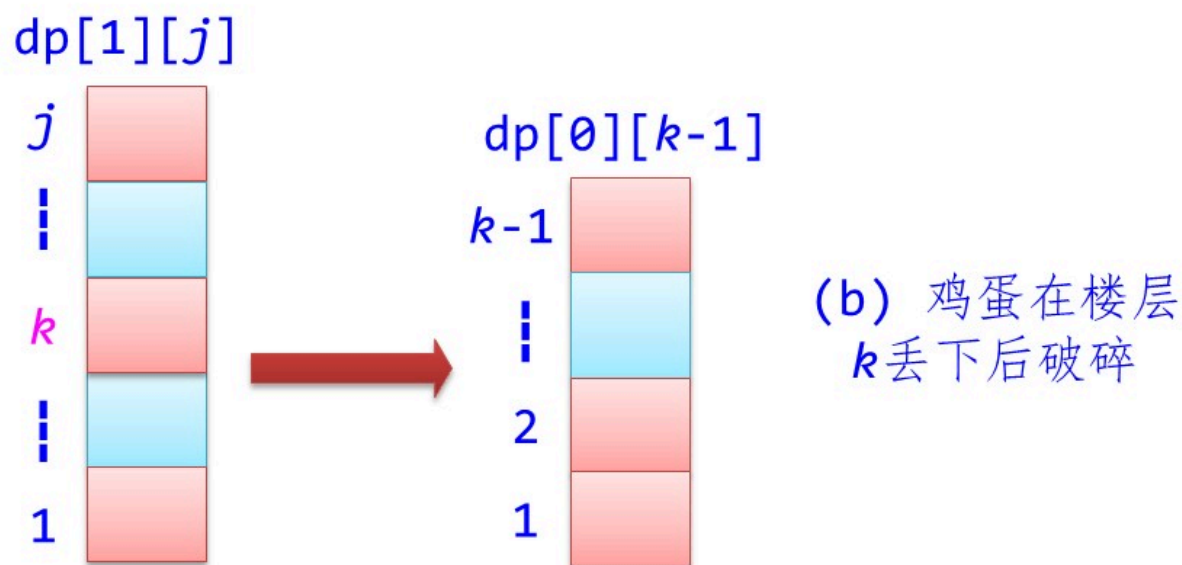


(a) 只剩一枚鸡蛋：从楼层1开始逐层验证

42/182

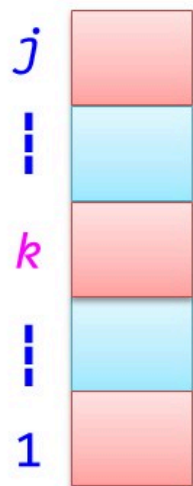
(2) 当 $i=1$ 时表示剩两枚鸡蛋，对于任意 j ($1 \leq j \leq n$)，第一次操作可以选择在 $1 \sim j$ 范围内的任一楼层 k ：

① 如果鸡蛋在楼层 k 丢下后破碎，接下来问题转化成 $i=0$ 时验证楼层 $1 \sim k-1$ 共 $k-1$ 层需要的次数即 $dp[0][k-1]$ ，如图 (b) 所示，总操作次数为 $dp[0][k-1]+1$ 。

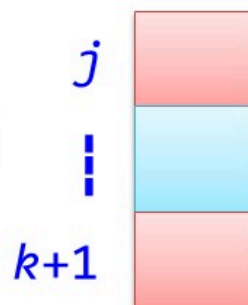


②如果鸡蛋在楼层 k 丢下后没碎，接下来问题转化成 $i=1$ 时验证楼层 $k+1 \sim j$ 共 $j-k$ 层需要的次数即 $dp[1][j-k]$ ，如图(c)所示，总操作次数为 $dp[1][j-k]+1$ 。

$dp[1][j]$



$dp[1][j-k]$



(c) 鸡蛋在楼层 k
丢下后没有碎

考虑最坏的情况是两者取最大值，同时 dp 元素取所有情况下的最小值即：

$$dp[1][j] = \min \left\{ \max_{1 \leq k \leq j} \{ dp[0][k-1] + 1, dp[1][j-k] + 1 \} \right\}$$

在求出 dp 数组后，显然 $dp[1][n]$ 就是找到 $1 \sim n$ 中某个楼层 f 的最少操作次数，返回该元素即可。

```

class Solution {
    const int INF=0x3f3f3f3f;           //表示 $\infty$ 
public:
    int twoEggDrop(int n) {
        int dp[2][n+1];
        memset(dp,0x3f,sizeof(dp));    //dp元素初始化为 $\infty$ 
        dp[0][0]=dp[1][0]=0;
        for (int j=1;j<=n;j++)
            dp[0][j]=j;
        for (int j=1;j<=n;j++) {
            for (int k=1;k<=j;k++)
                dp[1][j]=min(dp[1][j],
                    max(dp[0][k-1]+1,dp[1][j-k]+1));
        }
        return dp[1][n];
    }
};

```

46/182

EXE 执行用时为 **60ms**，内存消耗为 **5.8MB**。

进一步优化空间：

- 将 $dp[0][k]$ 表示为 k （因为只剩一枚鸡蛋时总有 $dp[0][k]=k$ ），这样有 $dp[0][k-1]+1=k-1+1=k$ 。
- 同时将 $dp[1][j]$ 表示为 $dp[j]$ 。
- 从而将二维数组 dp 优化为一维数组。

47/182

```

class Solution {
    const int INF=0x3f3f3f3f;           //表示∞
public:
    int twoEggDrop(int n) {
        int dp[n+1];
        memset(dp,0x3f,sizeof(dp));    //dp元素初始化为∞
        dp[0]=0;
        for (int j=1;j<=n;j++) {
            for (int k=1;k<=j;k++)
                dp[j]=min(dp[j],max(k,dp[j-k]+1));
        }
        return dp[n];
    }
};

```



执行用时为 **28ms**，内存消耗为 **5.7MB**。

48/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

49/182

7.10资源分配问题

 **问题描述：**资源分配问题是将数量一定的一种或若干种资源（原材料、资金、设备或劳动力等），合理地分配给若干使用者，使总收益最大。

例如，某公司有3个商店A、B、C，拟将新招聘的5名员工分配给这3个商店，各商店得到新员工后，每年的赢利情况如下表所示，求分配给各商店各多少员工才能使公司的赢利最大？

员工数 商店	0人	1人	2人	3人	4人	5人
A	0	3	7	9	12	13
B	0	5	10	11	11	11
C	0	4	6	11	12	12

50/182

解

- 设置二维动态规划数组 dp ，其中 $dp[i][s]$ 表示前 i 个商店（商店编号为 $0 \sim i-1$ ）共分配 s （ $0 \leq s \leq n$ ）名新员工的最优增收。
- 另外设置二维数组 $pnum$ ，其中 $pnum[i][s]$ 表示求出 $dp[i][s]$ 时对应商店 $i-1$ 的分配人数。
- 显然如果商店个数为 0 ，无论增加多少人员，总增收一定为 0 ，即 $dp[0][j]=0$ （ $0 \leq j \leq n$ ）。

51/182

- 现在考虑求 $dp[i][s]$ ，商店 $0 \sim i-1$ 共分配 s 名新员工。
- 商店 $i-1$ 理论上讲可以分配 j ($0 \leq j \leq s$) 名新员工，商店 $i-1$ 分配 j 名新员工时的增收为 $v[i-1][j]+dp[i-1][s-j]$ ，则最大增收为：

$$\max_{0 \leq s \leq n, 0 \leq j \leq s} \{ v[i-1][j] + dp[i-1][s-j] \}$$



$dp[0][j]=0$

边界条件 ($0 \leq j \leq n$)

$dp[i][s]=\max_{0 \leq s \leq n, 0 \leq j \leq s} \{ v[i-1][j] + dp[i-1][s-j] \}$ 其他

$pnum[i][s]=dp[i][s]$ 取最大值的 j



当求出 dp 数组后， $dp[m][n]$ 就是最终的最优增收。

52/182

```
vector<vector<int>> dp;           //二维动态规划数组
vector<vector<int>> pnum;        //分配人数

int plan(int m,int n,vector<vector<int>>&v) { //求dp和pnum
    dp=vector<vector<int>>(m+1,vector<int>(n+1,0));
    pnum=vector<vector<int>>(m+1,vector<int>(n+1,0));
    for (int j=0;j<=n;j++)      //置边界条件
        dp[0][j]=0;
```

53/182

```

for (int i=1;i<=m;i++) {
    for (int s=0;s<=n;s++) {
        int maxf=0,maxj=0;
        for (int j=0;j<=s;j++) {
            if ((v[i-1][j]+dp[i-1][s-j])>maxf) {
                maxf=v[i-1][j]+dp[i-1][s-j];
                maxj=j;
            }
        }
        dp[i][s]=maxf;
        pnum[i][s]=maxj;
    }
}
return dp[m][n];
}

```

可以从`pnum[m][n]`开始推导出各个商店分配的人数，用`vector<int>`容器`x`存放各个商店分配的人数。

```
vector<int> getx(int m,int n) {           //求一个最优分配方案
    vector<int> x;                        //存放一个最优分配方案
    int s=pnum[m][n];
    x.push_back(s);
    int r=n-s;                            //r为余下的人数
    for (int k=m;k>1;k--) {
        s=pnum[k-1][r];                  //求下一个阶段分配的人数
        x.push_back(s);
        r=r-s;                           //余下人数递减
    }
    reverse(x.begin(),x.end());           //逆置x
    return x;
}
```

55/182



【例7.5】盈利计划（LintCode1607★★★★）。共有 g ($1 \leq g \leq 100$) 名人员计划完成 m ($1 \leq m \leq 100$) 个活动，给出两个长度均为 m 的数组 `groups` ($1 \leq \text{group}[i] \leq 100$) 和 `profit` ($0 \leq \text{profit}[i] \leq 100$)，它们的含义是活动 i 需要投入 `groups[i]` 个人员可以获得 `profit[i]` 的盈利。每个人只能同时参加一个活动。为了获得至少 p 的盈利，求有多少种方案可以选择。因为答案很大，所以答案要对 10^9+7 取模。

例如， $g=5$ ， $p=3$ ，`group={2, 2}`，`profit={2, 3}`，答案为2，解释如下，共有两个活动，5个人，要产生至少3的盈利，方案1是参加全部活动（需要的人数为4），盈利为5；方案2是参加活动1（需要的人数为2），盈利为3。

要求设计如下成员函数：

```
int profitableSchemes(int g,int p,vector<int> &group,
                    vector<int> &profit) { }
```

56/182

解

- 假设 m 个活动的编号为 $0 \sim m-1$ ，设置三维动态规划数组 dp ，其中 $dp[i][s][j]$ 表示在前 i 个活动中选择了 s 个人并且盈利至少为 j 的方案总数。
- 在不考虑取模运算的情况下，满足要求的方案数 ans 为：

$$\sum_{s=0}^g dp[m][s][p]$$

57/182

状态转移方程

$dp[i][s][j] = dp[i-1][s][j]$ 当 $s < group[i-1]$

$dp[i][s][j] = dp[i-1][s][j] +$
 $dp[i-1][s - group[i-1]][\max(0, j - profit[i-1])]$ 其他

不做活动 $i-1$

做活动 $i-1$

当求出 dp 数组后，由其计算出 ans 并且返回 ans 即可，在计算中考虑对 10^9+7 取模的情况。

58/182

```
class Solution {  
    const int MOD=(int)1e9+7;  
public:  
    int profitableSchemes(int g,int p,vector<int> &group,  
        vector<int> &profit) {  
        int m=group.size() ;  
        vector<vector<vector<int>>> dp(m+1,  
            vector<vector<int>>(g+1,vector<int>(p+1,0)));  
        dp[0][0][0]=1;  
    }  
};
```

59/182

```

for (int i=1;i<=m;i++) {
    for (int s=0;s<=g;s++) {
        for (int j=0;j<=p;j++) {
            if (s<group[i-1])
                dp[i][s][j]=dp[i-1][s][j];
            else
                dp[i][s][j]=(dp[i-1][s][j]+
                dp[i-1][s-group[i-1]][max(0,j-profit[i-1])]) % MOD;
        }
    }
}

int ans=0;                                //存放答案
for (int s=0;s<=g;s++)
    ans=(ans+dp[m][s][p]) % MOD;
return ans;
}
};

```

EXE

执行用时为**61ms**，内存消耗为**5.68MB**。

60/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

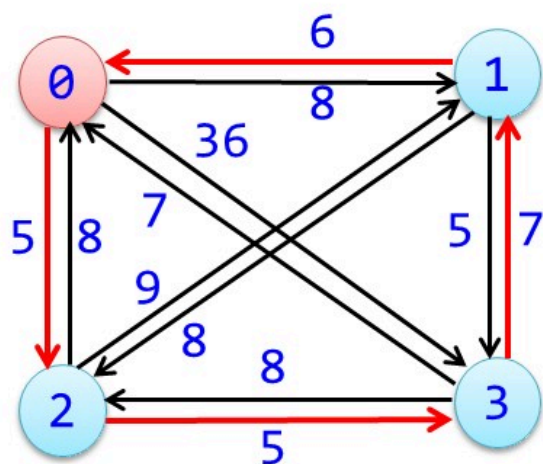
7.7 0/1背包问题

7.13矩阵连乘问题

61/182

7.11 旅行商问题

 **问题描述：** 假设有一个旅行商要拜访 n 个城市，他必须选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市，要求路径长度最短的路径。



路径1: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$: 28

路径2: $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$: 29

路径3: $0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 0$: 26

路径4: $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$: 23

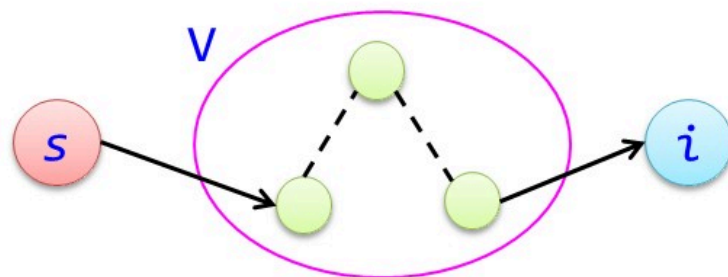
路径5: $0 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$: 59

路径6: $0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 0$: 59

62/182

解

设 $f(V, i)$ 表示从顶点 s 出发经过 V （一个顶点的集合）中全部顶点（每个顶点恰好经过一次）到达顶点 i 的最短路径长度。



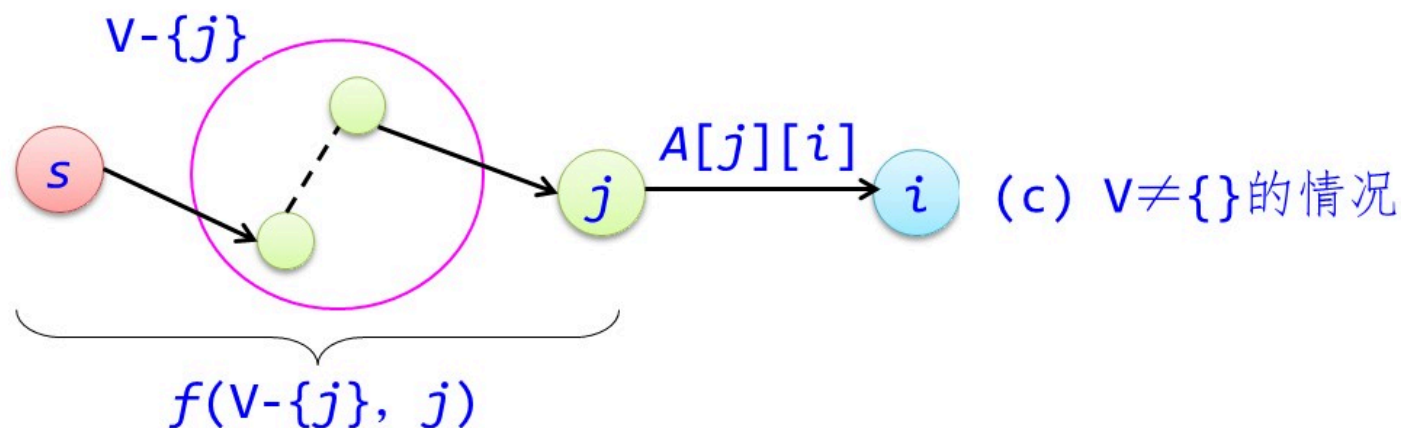
(a) $f(V, i)$ 的含义

- 当 V 为空集, 那么 $f(V, i)$ 表示从顶点 s 不经过任何顶点到达顶点 i , 显然此时有 $f(V, i) = A[s][i]$ 。



- 如果 V 不为空, 对于 $j \in V$, 那么 $f(V - \{j\}, j)$ 就是子问题, 尝试 V 中的每个顶点 j 对应的子问题, 则

$$f(V, i) = \min_{j \in V} \{f(V - \{j\}, j) + A[j][i]\}$$



- 这里假设起点 $s=0$ ，操作过程改为从 V 中取出顶点 i ，置 $V1=V-\{i\}$ ，尝试 $V1$ 中的每个顶点 j 作为一个子问题，所以递归出口是 V 中只有一个顶点 i 时对应的路径长度为 $A[0][i]$ 。
- 设计二维动态规划数组 dp ，其中 $dp[V][i]$ 表示从起点 0 出发经过 V 中的全部顶点到达顶点 i ($i \in V$) 的最短路径长度。
- 首先 dp 所有元素初始化为 ∞ ，当 V 中只有一个顶点 i 时置 $dp[V][i]=A[0][i]$ 。

状态转移方程

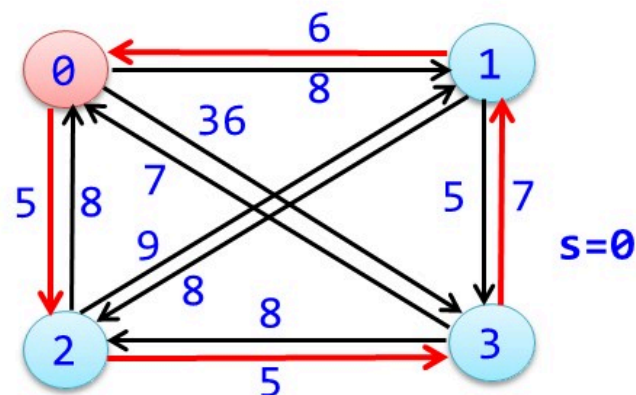
$$dp[V][i] = A[0][i]$$

当 $V = \{i\}$ 时 ($1 \leq i < n$)

$$dp[V][i] = \min_{i \in V \text{ 且 } j \in V} \{dp[V - \{i\}][j] + A[j][i]\} \quad \text{其他}$$

- 将 V 改为十进制数，在对应的二进制数中用位序 i 的值表示是否包含顶点 $i+1$ 。
- 例如， $V=11$ ，对应的二进制数是 $[1011]_2$ ，位序 0 的位值是 1 说明包含顶点 1 ，位序 1 的位值是 1 说明包含顶点 2 ，位序 3 的位值是 1 说明包含顶点 4 ， V 表示的顶点集合是 $\{1, 2, 4\}$ 。

通过 V 枚举所有的顶点集即 V 从 0 到 $2^{n-1}-1$ ，从而求出 dp 数组，那么 $\min\{dp[\{1, 2, \dots, n-1\}][i]+A[i][0]\}$ 即为所求。



V 从 0 到 7 循环：

① $V=0=[000]_2$ ，即 $V=\{\}$ ，没有考虑任何顶点。

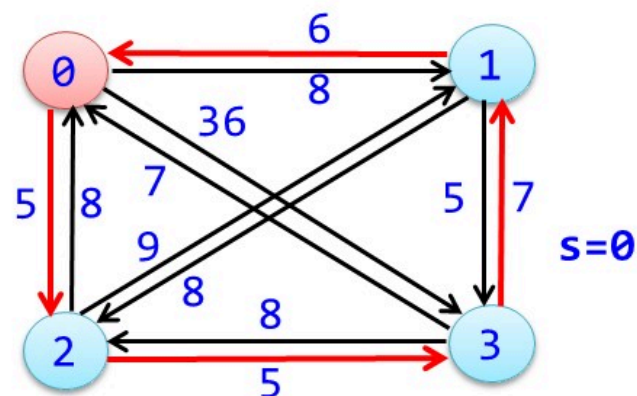
② $V=1=[001]_2$ ，即 $V=\{1\}$

$i=1 \Rightarrow V1=\{\}$ ， $dp[V1][1]=A[0][1]=8$

③ $V=2=[010]_2$ ，即 $V=\{2\}$

$i=2 \Rightarrow V1=\{\}$ ， $dp[V1][2]=A[0][2]=5$

67/182



④ $V=3=[011]_2$, 即 $V=\{1, 2\}$

$i=1 \Rightarrow V1=\{2\}$: $j=2$, $dp[V1][1]=\min\{dp[2][2]+A[2][1]\}=14$

$i=2 \Rightarrow V1=\{1\}$, $j=1$, $dp[V1][2]=\min\{dp[1][1]+A[1][2]\}=16$

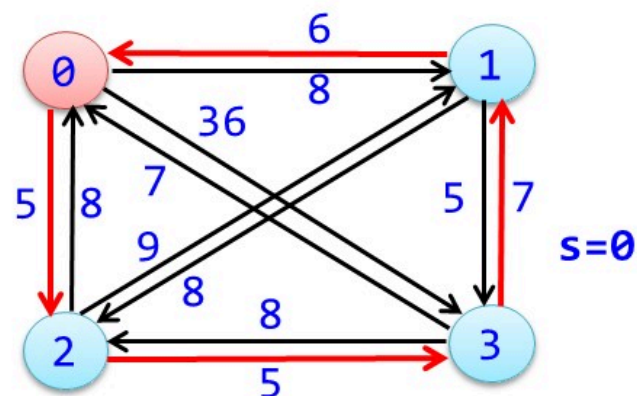
⑤ $V=4=[100]_2$, 即 $V=\{3\}$

$i=3 \Rightarrow V1=\{\}$, $dp[V1][3]=A[0][3]=36$

⑥ $V=5=[101]_2$, 即 $V=\{1, 3\}$

$i=1 \Rightarrow V1=\{3\}$, $j=3$, $dp[V1][1]=\min\{dp[4][3]+A[3][1]\}=43$

$i=3 \Rightarrow V1=\{1\}$, $j=1$, $dp[V1][3]=\min\{dp[1][1]+A[1][3]\}=13$



⑦ $V=6=[110]_2$, 即 $V=\{2, 3\}$

$i=2 \Rightarrow V1=\{3\}, j=3, dp[V1][2]=\min\{dp[4][3]+A[3][2]\}=44$

$i=3 \Rightarrow V1=\{2\}, j=2, dp[V1][3]=\min\{dp[2][2]+A[2][3]\}=10$

⑧ $V=7=[111]_2$, 即 $V=\{1, 2, 3\}$

$i=1 \Rightarrow V1=\{2, 3\}, j=2, dp[V1][1]=\min\{dp[6][2]+A[2][1]\}=53$

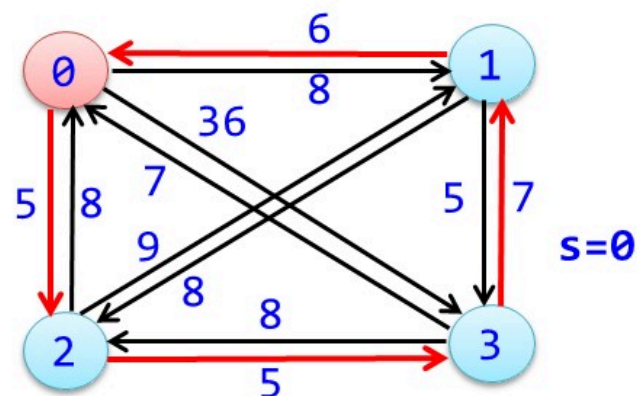
$i=1 \Rightarrow V1=\{2, 3\}, j=3, dp[V1][1]=\min\{dp[6][3]+A[3][1]\}=17$

$i=2 \Rightarrow V1=\{1, 3\}, j=1, dp[V1][2]=\min\{dp[5][1]+A[1][2]\}=51$

$i=2 \Rightarrow V1=\{1, 3\}, j=3, dp[V1][2]=\min\{dp[5][3]+A[3][2]\}=21$

$i=3 \Rightarrow V1=\{1, 2\}, j=1, dp[V1][3]=\min\{dp[3][1]+A[1][3]\}=19$

$i=3 \Rightarrow V1=\{1, 2\}, j=2, dp[V1][3]=\min\{dp[3][2]+A[2][3]\}=19$



算法执行的最终结果 ($V=[111]_2=7$) 为

$\text{ans}=\min\{\text{dp}[7][1]+A[1][0], \text{dp}[7][2]+A[2][0], \text{dp}[7][3]+A[3][0]\}$
 $=\{23, 29, 26\}=23。$

70/182

```
bool inset(int V,int j) {           //判断顶点j是否在used中
    return (V & (1<<(j-1)))!=0;
}

int delj(int V,int j) {             //从V中删除顶点i得到V1
    return V^(1<<(j-1));
}

vector<vector<int>> dp;              //二维动态规划数组
```

71/182

```
int TSP(vector<vector<int>> &A) { //求TSP所有解的路径长度
    int n=A.size();                //n为顶点个数减1（除开起点0）
    dp=vector<vector<int>>(1<<n,vector<int>(n,INF));
    dp[0][0]=0;
```

```

for(int V=1;V<(1<<(n-1));V++) {
    for(int i=1;i<n;i++) {           //顶点i从1到n-1循环
        if(inset(V,i)) {             //顶点i在V中
            if(V==(1<<(i-1))) {       //V中只有一个顶点i
                dp[V][i]=min(dp[V][i],A[0][i]);
            }
            else {                     //V中有多个顶点
                int V1=delj(V,i);      //从V中删除顶点i得到V1
                for(int j=1;j<n;j++) {
                    if(inset(V1,j)) { //处理V1中每一个顶点j
                        dp[V][i]=min(dp[V][i],dp[V1][j]+A[j][i]);
                    }
                }
            }
        }
    }
}

```

73/182

```
int ans=INF;
for(int i=1;i<n;i++)           //求TSP路径长度
    ans=min(ans,dp[(1<<(n-1))-1][i]+A[i][0]);
return ans;
}
```

TSP算法分析：若图中有 n 个顶点，该算法主要含3重循环，第一重循环需要操作 $\{1, 2, \dots, n-1\}$ 的每个子集，执行时间为 $O(2^{n-1})$ ，后面两重循环的执行时间为 $O(n^2)$ ，所以算法的时间复杂度为 $O(n^2 \times 2^n)$ 。

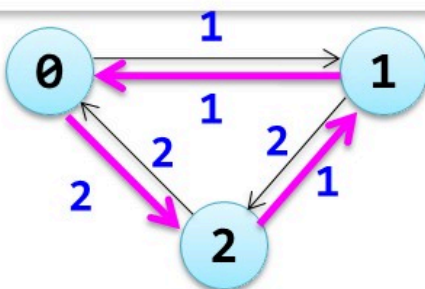


【例7.6】=【例3.9】旅行计划

(LintCode1891★★) 有 n ($n \leq 10$) 个城市，城市编号为 $0 \sim n-1$ 。给出邻接矩阵 arr 代表任意两个城市的距离，其中 $arr[i][j]$ ($arr[i][j] \leq 10000$) 表示从城市 i 到城市 j 的距离。**Alice**在周末制定了一个游玩计划，她从所在的城市 0 开始，游玩其他的 $1 \sim n-1$ 的全部城市，最后回到城市 0 。

Alice想知道她能完成游玩计划需要行走的最小距离，返回这个最小距离。除了城市 0 之外每个城市都只能经过一次，且城市 0 只能是起点和终点，**Alice**中途不可经过城市 0 。

例如， $arr = \{\{0, 1, 2\}, \{1, 0, 2\}, \{2, 1, 0\}\}$ ，答案是4，对应的路径是城市 $0 \rightarrow$ 城市 $2 \rightarrow$ 城市 $1 \rightarrow$ 城市 0 。



75/182

解

- 采用与前面求解旅行商问题完全相同的思路，设置起始顶点 $s=0$ 。

状态转移方程

$$\begin{aligned} dp[V][i] &= A[0][i] && \text{当 } V=\{i\} \text{ 时 } (1 \leq i < n) \\ dp[V][i] &= \min_{i \in V \text{ 且 } j \in V} \{dp[V-\{i\}][j] + A[j][i]\} && \text{其他} \end{aligned}$$

76/182

```
class Solution {  
    const int INF=0x3f3f3f3f;           //表示∞  
public:  
    int travelPlan(vector<vector<int>> &arr) {  
        return TSP(arr);  
    }  
}
```

77/182

```
int TSP(vector<vector<int>> &A) {    //求TSP问题（起始点为0）
    int n=A.size();
    vector<vector<int>> dp;          //二维动态规划数组
    dp=vector<vector<int>>(1<<n,vector<int>(n,INF));
                                     //均设置为 $\infty$ 

    dp[0][0]=0;
```

78/182

```

for(int V=0;V<(1<<(n-1));V++) {
    for(int i=1;i<n;i++) {           //顶点i从1到n-1循环
        if(inset(V,i)) {           //顶点i在V中
            if(V==(1<<(i-1))) {     //V中只有一个顶点i
                dp[V][i]=min(dp[V][i],A[0][i]);
            }
            else {                  //V中有多个顶点
                int V1=delj(V,i);    //从V中删除顶点i得到V1
                for(int j=1;j<n;j++) {
                    if(inset(V1,j)) { //顶点j在V1中
                        dp[V][i]=min(dp[V][i],dp[V1][j]+A[j][i]);
                    }
                }
            }
        }
    }
}

```

79/182

```
int ans=INF;  
for(int i=1;i<n;i++)           //求答案  
    ans=min(ans,dp[(1<<(n-1))-1][i]+A[i][0]);  
return ans;  
}  
};
```



执行用时为**41ms**，内存消耗为**4.25MB**。

80/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

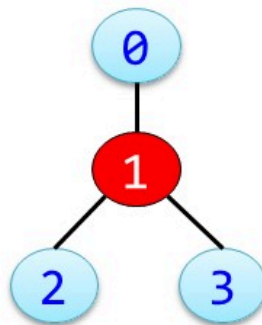
81/182



7.12 最少士兵数问题

问题描述 (POJ1463, 时间限制为2000ms, 内存限制为10000KB): 有一座中世纪的城市, 其道路构成一棵树, 为了保卫城市, 必须在道路结点上放置一定数量的士兵 (每个结点处最多只能放置一个士兵), 以便他们可以观察所有边缘。给定一棵树, 求需要放置的最少士兵数。

例如, 对于如图所示的一棵树, 答案为1, 即最少在结点1放置一个士兵, 这样可以观察所有边缘。



82/182

输入格式：输入包含多个测试用例。每个测试用例代表一棵树，其描述如下：

第一行是结点数 n ($0 < n \leq 1500$ ， n 个结点编号为 $0 \sim n-1$)，接下来 n 行，每行的格式为：

结点编号(子结点数 m) 子结点1编号 子结点2编号... 子结点 m 编号

或者

结点编号(子结点数 0)

每行输入中的子结点数不会超过 10 ，每条边在输入数据中只出现一次。

输出格式：对于每个测试用例输出一行，其中包含一个表示最少士兵数的整数。

输入样例:

4

0:(1) 1

1:(2) 2 3

2:(0)

3:(0)

5

3:(3) 1 4 2

1:(1) 0

2:(0)

0:(0)

4:(0)

输出样例:

1

2

84/182

解

- 城市由 n 个结点构成，编号分别为 $0 \sim n-1$ ，给出若干连接结点的道路，整个城市构成一棵树结构（有根树）。
- 城市图采用邻接表 G 存储，即 $G[i]$ ($0 \leq i \leq n-1$) 存放顶点 i 的全部子结点的编号。
- 为了求出哪个结点是根结点，设置一个 $parent$ 数组，其中 $parent[i]=j$ 表示结点 i 的双亲为 j ，初始时 $parent$ 数组元素均置为 -1 ，当输入结束后，从任意一个结点出发找到双亲，直到 $parent[root]$ 为 -1 ，则 $root$ 就是根结点。

85/182

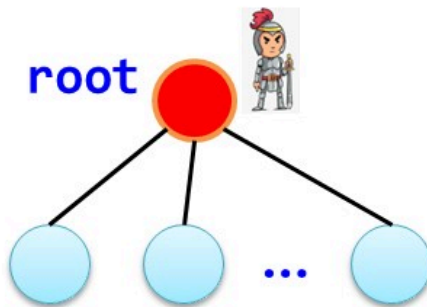
树形动态规划

- 设置二维动态规划数组 dp 。
- $dp[i][1]$ 表示在结点 i 上放置一个士兵时以该结点为根的子树所需的最少士兵数。
- $dp[i][0]$ 表示在结点 i 上不放置士兵时以该结点为根的子树所需的最少士兵数。
- 对于以 $root$ 为根的树，采用深度优先搜索方式，先求出其所有子树的 dp ，考虑根结点 $root$ 的两种情况。

86/182

- 若在根结点 **root** 上放置一个士兵，该士兵可以观察到根结点 **root** 到每个子结点的边缘，如图 (a) 所示，则它的每一个子结点可以放置一个士兵，也可以不放置士兵，两种子情况中取最小值，即：

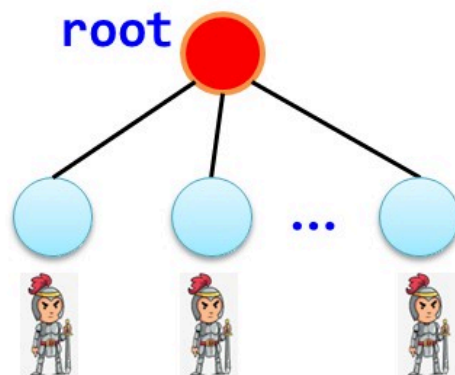
$$dp[root][1] = 1 + \sum_{son \in G[root]} \min \{dp[son][1], dp[son][0]\}$$



(a) root 放置一个士兵

- 若在根结点 $root$ 上不放置士兵，则它的每一个子结点必须放置一个士兵，这样才能从每个子结点位置的士兵观察到该子结点到根结点 $root$ 的边缘，如图(b)所示，即：

$$dp[root][0] = 0 + \sum_{son \in G[root]} dp[son][1]$$



(b) $root$ 不放置士兵

- 在求出 dp 数组后，则 $\min(dp[root][0], dp[root][1])$ 就是以 $root$ 结点为根的树中最少士兵数，返回该结果即可。

```
#include<iostream>
#include<vector>
#include<cstring>
using namespace std;
const int MAXN=1510;
vector<int> G[MAXN];           //邻接表
int parent[MAXN];            //双亲数组
int dp[MAXN][2];             //二维动态规划数组
```

89/182

```

void dfs(int root) {                                     //深度优先搜索
    int m=G[root].size();
    for(int i=0;i<m;i++) {
        dfs(G[root][i]);
    }
    dp[root][0]=0;
    dp[root][1]=1;
    for(int i=0;i<m;i++) {
        dp[root][1]+=min(dp[G[root][i]][1],dp[G[root][i]][0]);
        dp[root][0]+=dp[G[root][i]][1];
    }
}

```

$$dp[root][1] = 1 + \sum_{son \in G[root]} \min \{dp[son][1], dp[son][0]\}$$

$$dp[root][0] = 0 + \sum_{son \in G[root]} dp[son][1]$$

90/182

```

int main() {
    int n,m,fa,son;
    while(scanf("%d",&n)!=EOF) {
        memset(parent,0xff,sizeof(parent));    //初始化为-1
        for(int i=1;i<=n;i++) {
            scanf("%d:(%d",&fa,&m);
            G[fa].clear();
            while(m--) {
                scanf("%d",&son);
                G[fa].push_back(son) ;
                parent[son]=fa;    //置son的双亲为fa
            }
        }
    }
}

```

fa

son1 son2 ...

邻接表G

91/182

```
int root=1;
while(parent[root]!=-1)    //找到根结点的编号root
    root=parent[root];
dfs(root);
int ans=min(dp[root][0],dp[root][1]);
printf("%d\n",ans);
}
return 0;
}
```



执行用时为**375ms**，内存消耗为**272KB**。

92/182

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

93/182

7.13 矩阵连乘问题

 **问题描述：** 假设 A 是 $p \times q$ 矩阵， B 是 $q \times r$ 矩阵，在计算 $C=A \times B$ 的标准算法中需要做 pqr 次数乘。

给定 n ($n > 2$) 个矩阵 $A_1, A_2, \dots, A_n$ ，其中 A_i 和 A_{i+1} 是可乘的 ($1 \leq i \leq n-1$)，例如， $n=3$ 时， A_1, A_2 和 A_3 的大小分别为 10×100 、 100×50 和 50×5 。

求这 n 个矩阵的乘积 $A_1 \times A_2 \times \dots \times A_n$ 时最少的数乘次数是多少？

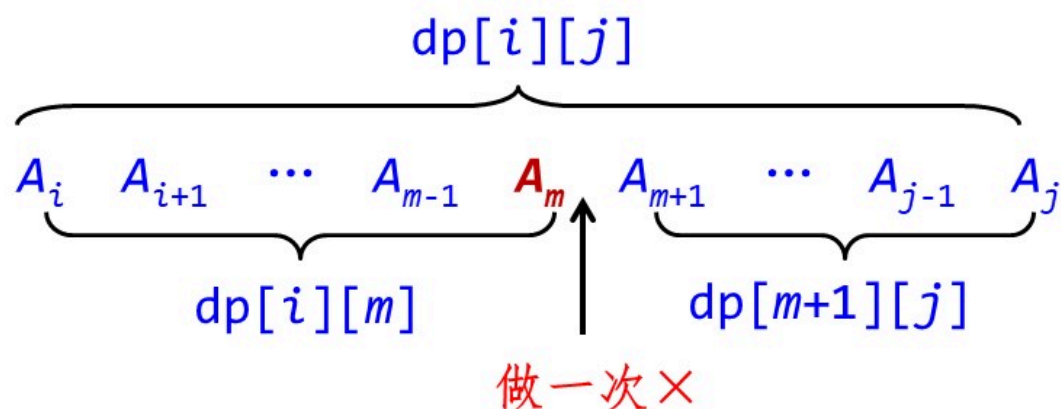
解

区间动态规划

- 用一维数组 $p[0..n]$ 表示 n 个矩阵的大小, $p[0]$ 表示 A_1 的行数, $p[i]$ 表示 A_i ($1 \leq i \leq n$) 的列数, 例如 $n=3$, A_1 、 A_2 和 A_3 的大小分别为 10×100 、 100×50 和 50×5 , 对应的 $p[0..3] = \{10, 100, 50, 5\}$ 。
- 其运算方式有 $((A_1 \times A_2) \times A_3)$ (总计数乘次数为 52500) 和 $(A_1 \times (A_2 \times A_3))$ (总计数乘次数为 30000)。
- 实际上每加上一个括号就是做一次两个矩阵相乘的标准算法, 用 $A[i..j]$ 表示 $A_i \times \cdots \times A_j$ 的连乘结果, 显然 $A[i..j]$ 是一个 $p[i-1] \times p[j]$ 大小的矩阵。
- 设计二维动态规划数组 $dp[n+1][n+1]$, $dp[i][j]$ 表示计算 $A[i..j]$ 的最少数乘次数, 初始时置 dp 所有元素为 0。

95/182

- 用 len 枚举区间 $[i..j]$ (len 从 2 开始递增)，如下图所示，枚举该区间中每个位置 m (m 称为分割点， $i \leq m < j-1$)。



- 本次矩阵相乘的数乘次数为 $p[i-1] \times p[m] \times p[j]$ ，则有
 $dp[i][j] = \min_{i \leq m < j} \{ dp[i][m] + dp[m+1][j] + p[i-1] * p[m] * p[j] \}$

- 另外设计一个二维数组 $s[n+1][n+1]$ ，其中 $s[i][j]$ 表示 $dp[i][j]$ 取最小值时的分割点。
- 当求出 dp 数组后， $dp[1][n]$ 就是最少的数乘次数。

```

vector<vector<int>> dp;           //二维动态规划数组
vector<vector<int>> s;           //存放最优分割点

void matrixchain(int p[],int n) { //求dp和s
    for (int len=2;len<=n;len++) { //按长度len枚举区间[i,j]
        for (int i=1;i+len-1<=n;i++) {
            int j=i+len-1;
            dp[i][j]=dp[i+1][j]+p[i-1]*p[i]*p[j];
            s[i][j]=i;
            for (int m=i+1;m<j;m++) { //枚举分割点m(不包含i和j)
                int tmp=dp[i][m]+dp[m+1][j]+p[i-1]*p[m]*p[j];
                if(tmp<dp[i][j]) {
                    dp[i][j]=tmp;
                    s[i][j]=m;
                }
            }
        }
    }
}

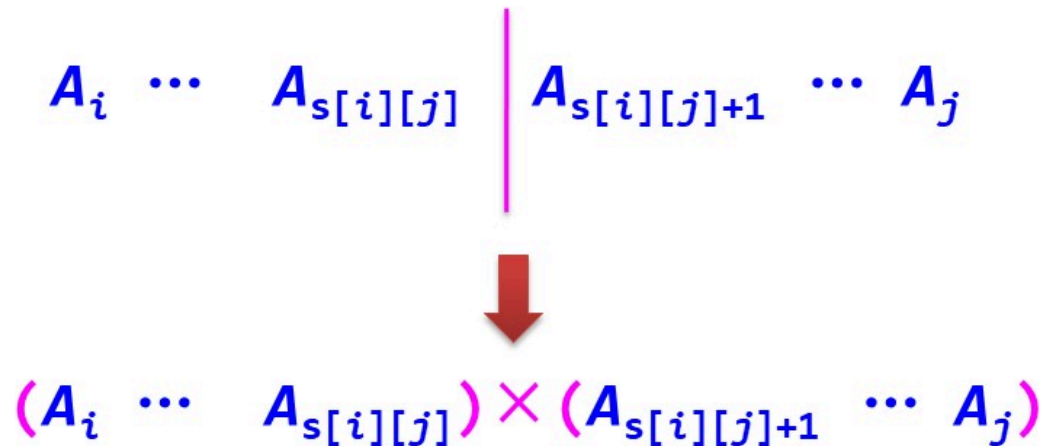
```

98/182

```

void getx(int i,int j) {           //构造一个最优解
    if(i==j) return;
    getx(i,s[i][j]);
    getx(s[i][j]+1,j);
    printf("  A[%d..%d] × A[%d..%d]\n",i,s[i][j],s[i][j]+1,j);
}

```



```

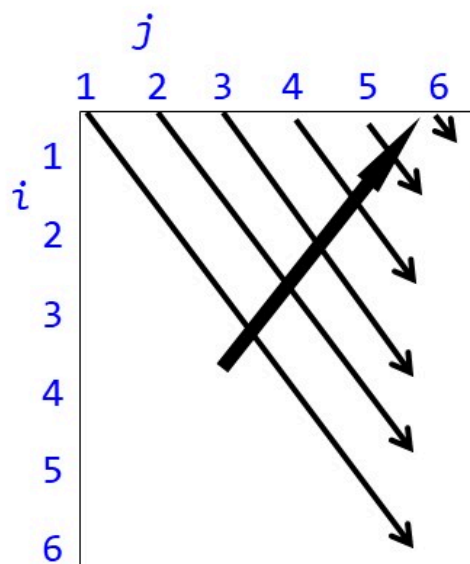
void solve(int p[],int n) {                                //求解算法
    dp=vector<vector<int>>(n+1,vector<int>(n+1,0));
    s=vector<vector<int>>(n+1,vector<int>(n+1));
    matrixchain(p,n);
    printf("矩阵最优计算次序:\n");
    getx(1,n);
    printf("最少数乘次数=%d\n",dp[1][n]);
}

```

 **matrixchain**算法分析：该算法中包含3重循环，时间复杂度为 $O(n^3)$ ，空间复杂度为 $O(n^2)$ 。



$n=6$, 矩阵分别是 $A_1[20 \times 25]$ 、 $A_2[25 \times 5]$ 、 $A_3[5 \times 15]$ 、 $A_4[15 \times 10]$ 、 $A_5[10 \times 20]$ 、 $A_6[20 \times 25]$



(a) dp计算顺序

	j					
	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

(b) dp的结果

	j					
	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

(c) s的结果



枚举区间还有其他方式吗？

102/182